COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 - DATA STRUCTURES AND ALGORITHMS | **SECTION:** | CPE21S1 |
| **DATE PERFORMED:** | Oct 13, 2026 | **DATE SUBMITTED:** | Oct 15, 2026 |
| **NAME:** | Corro, Yazinel<br>Obias, Jave Joaquin | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**ACTIVITY NO. 12.1 [Algorithmic Strategies]**

## 1. PROCEDURE OUTPUT

**ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)**

| Strategy | Algorithm | Analysis |
|---|---|---|
| Recursion | Merge Sort | The function repeatedly calls itself on smaller sub-arrays until reaching the base case where n is less than or equal to 1. This recursive reduction breaks down the unsorted list into single-element sub-lists before merging them back together. |
| Brute Force | Linear Search | It checks every element in the array sequentially from index 0 to n-1 without using any shortcuts. This exhaustive comparison guarantees finding the target by trying all possible positions one by one. |
| Backtracking | Depth-First Search | It explores a branch as deeply as possible until hitting a dead end or invalid state. Once blocked, it backtracks by returning up the call stack to systematically test alternative paths. |
| Greedy | Selection Sort | At each iteration, it scans the unsorted portion of the list to find the minimum element. It then greedily swaps this minimum into its correct position at the end of the sorted sub-array. |

| | | |
|---|---|---|
| Divide and Conquer | Quick Sort | It divides the array into two sub-arrays by selecting a pivot and partitioning the elements around it. It then recursively conquers and sorts those sub-arrays, which combines the entire array in-place. |

**Table 12-1. Algorithmic Strategies and Examples**

**Code and Output**

ilob > 12.2 > memoi.cpp > getMinSteps(int)

```cpp
#include <iostream>
#include <algorithm>

int memo[101];

int getMinSteps(int n) {
    if (n == 1) return 0;
    if (memo[n] != -1) return memo[n];

    int r = 1 + getMinSteps(n - 1);

    if (n % 2 == 0)
        r = std::min(r, 1 + getMinSteps(n / 2));

    if (n % 3 == 0)
        r = std::min(r, 1 + getMinSteps(n / 3));

    return memo[n] = r;
}

int main() {
    std::fill(memo, memo + 101, -1);

    int n = 10;
    std::cout << "Input: " << n << std::endl;
    std::cout << "Minimum steps for n: " << getMinSteps(n) << std::endl;

    return 0;
}
```

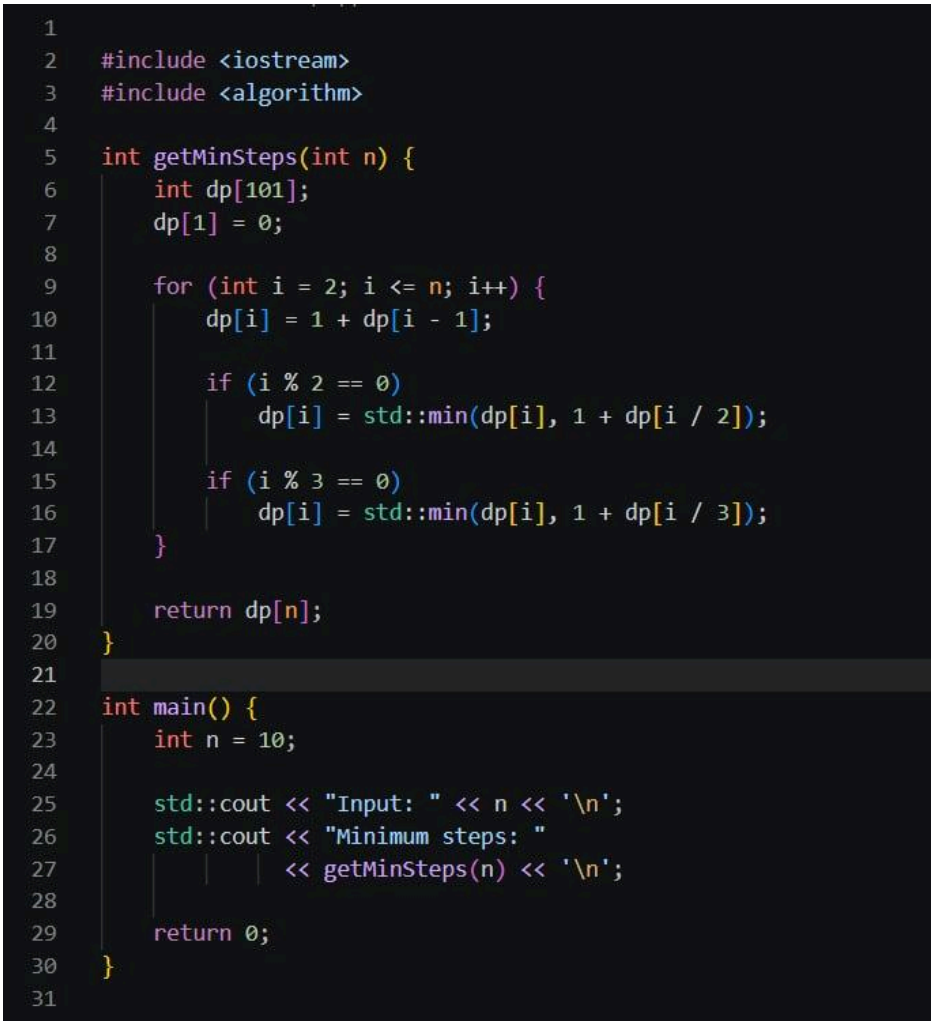
```
Input: 10
Minimum steps for n: 3
```

**Analysis**

| | |
|---|---|
| **Corro** | I observe that this program solves the problem top down with recursion and a memo array . I also get 3 steps for input of 10 . I learned that this recursive method breaks the big problem down and stores past answers in memory so the program never has to do the same calculation twice. |
| **Obias** | In this program, it uses recursion and memoization to find the fewest steps needed to reduce a number down to 1, the function "getMinSteps" checks if the answer is already in the array so it skips doing the same math over and over. Inside main, it sets up the array with -1, passes 10 for n, and runs the code to output 3. |

**Table 12-2. Memoization Implementation**

**Code and Output**



```cpp

#include <iostream>
#include <algorithm>

int getMinSteps(int n) {
    int dp[101];
    dp[1] = 0;

    for (int i = 2; i <= n; i++) {
        dp[i] = 1 + dp[i - 1];

        if (i % 2 == 0)
            dp[i] = std::min(dp[i], 1 + dp[i / 2]);

        if (i % 3 == 0)
            dp[i] = std::min(dp[i], 1 + dp[i / 3]);
    }

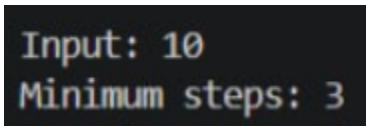
    return dp[n];
}

int main() {
    int n = 10;

    std::cout << "Input: " << n << '\n';
    std::cout << "Minimum steps: "
              << getMinSteps(n) << '\n';

    return 0;
}

```




| Analysis | |
|---|---|
| **Corro** | I noticed that this program solves the problem from bottom to top, with an iterative loop that fills an array with steps count, and successfully reaches 3 steps for an input of 10. I learned that this iterative method builds up the solutions sequentially from smallest values up to the target, avoiding duplicated computations efficiently. |
| **Obias** | In this program, it uses a bottom-up iterative loop with a DP table instead of recursion to solve the problem. Running it with an input of 10 gave me 3 steps too. It showed me how building up from smaller numbers lets you save past answers as you go, which cuts out unnecessary recursive calls while keeping it efficient. |

**Table 12-3. Bottom-Up Dynamic Programming Implementation**

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies**

**Pseudocode**

```
FUNCTION countPaths(matrix, i, j, remainingCost)
  IF i or j is outside matrix bounds
    RETURN 0
  END IF

  subtract matrix[i][j] from remainingCost

  IF current cell is destination
    IF remainingCost == 0
      RETURN 1
    ELSE
      RETURN 0
    END IF
  END IF

  IF remainingCost < 0
    RETURN 0
  END IF

  right = countPaths(matrix, i, j + 1, remainingCost)
```

```
    down = countPaths(matrix, i + 1, j, remainingCost)

    RETURN right + down

FUNCTION END

START
    Initialize 4x4 matrix
    Set targetCost = 25

    result = countPaths(matrix, 0, 0, targetCost)

    Print targetCost
    Print result
END
```

Code and Output

```cpp

#include <iostream>

const int M = 4;
const int N = 4;

// Recursive function to count paths with the given cost
int countPaths(int matrix[M][N], int i, int j, int remainingCost)
{
    // Check if the position is outside the matrix
    if (i >= M || j >= N)
        return 0;

    // Subtract the current cell's cost
    remainingCost -= matrix[i][j];

    // Check if the destination is reached
    if (i == M - 1 && j == N - 1)
    {
        return (remainingCost == 0) ? 1 : 0;
    }

    // Stop if the remaining cost becomes negative
    if (remainingCost < 0)
        return 0;

    // Recursively move right and down
    int right = countPaths(matrix, i, j + 1, remainingCost);
    int down = countPaths(matrix, i + 1, j, remainingCost);

    // Return the total number of valid paths
    return right + down;
}
```

```cpp
int main()
{
    int matrix[M][N] = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int targetCost = 25;

    int result = countPaths(matrix, 0, 0, targetCost);

    std::cout << "Target Cost: " << targetCost << std::endl;
    std::cout << "Number of valid paths: " << result << std::endl;

    return 0;
}
```

```
Target Cost: 25
Number of valid paths: 0
PS C:\Users\yazinel\OneDrive\Desk
```

| Analysis | |
|---|---|
| **Corro** | The program uses a recursive function to explore paths in a 4x4 grid, subtracting cell costs, to find valid paths that sum to a target cost of 25. I learned that recursion can be used to branch the function to test both right and down at each cell, with early quitting if the budget goes negative or out of bounds. Finally I saw from the output that the number of valid paths which meet this exact target cost for the grid given is 0 |
| **Obias** | In this program, the problem starts from the bottom up using an iterative loop and a DP array instead of recursion. I learned that building solutions progressively from smaller values avoids function overhead while keeping track of past answers. The output for an input of 10, the program correctly calculates and shows 3 steps. |
| **Conclusion** | |
| **Corro** | In conclusion, I've discovered that a variety of techniques, including recursion |

| | |
|---|---|
| | and dynamic programming, can be used to solve challenging computational issues. I learned how recursive branching effectively investigates grid-based paths and costs, and how top-down memoization and bottom-up loops effectively prevent repetitive work. In the end, these codes demonstrated to me how these algorithmic approaches deconstruct problems in order to find solutions or trace the paths in detail. |
| **Obias** | In conclusion, this lab activity helped me understand how recursion and dynamic programming handle tough coding problems. It showed me how recursive branching explores every available path, while memoization and loops prevent the code from doing extra calculations. Ultimately, seeing these techniques side by side made it much easier to understand how algorithms break down large problems to find the right path. |

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



**Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.**

***I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.***